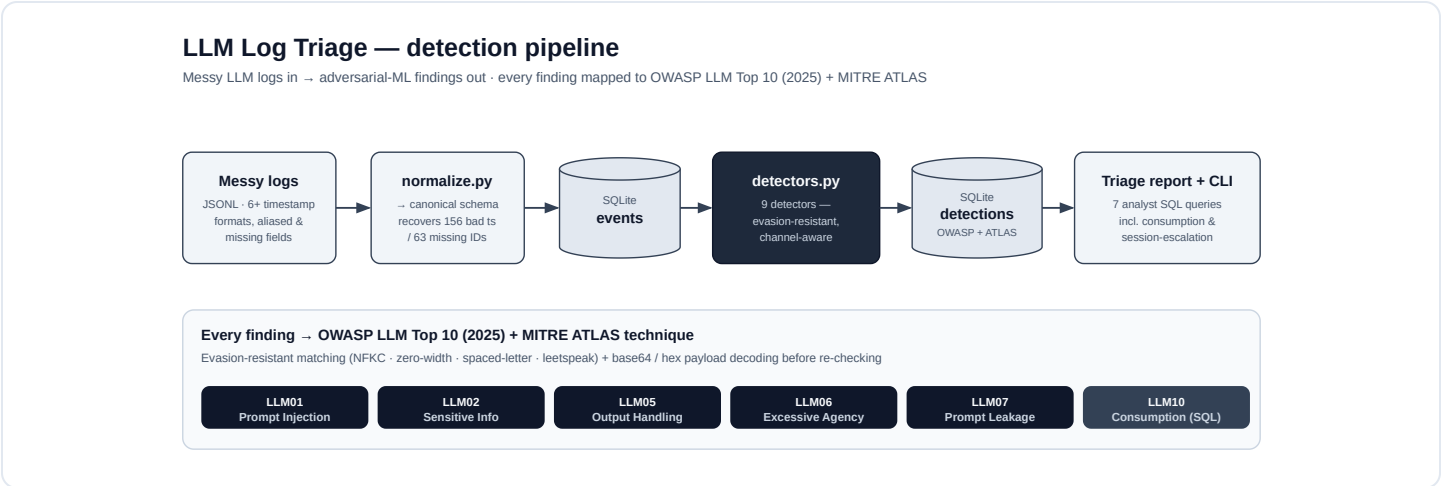


LLM Log Triage

An LLM threat-detection pipeline — Python + SQL applied to adversarial-ML log analysis

Izu Uhiara · Targeting: Technical Intelligence Analyst, Strategic Intelligence & Analysis (SIA), OpenAI · github.com/OMGstacks/openai-tia-prep

Frontier-AI threat intelligence means investigating how large language models break — prompt injection, jailbreaks, data exfiltration — not how networks break. This project does that work end to end: it ingests deliberately **messy** LLM interaction logs, normalizes them, flags adversarial activity with detectors **dual-mapped to the OWASP LLM Top 10 (2025) and MITRE ATLAS**, and answers investigative questions in SQL — the day-to-day of **surfacing novel harms and abuse patterns** and turning them into **structured, decision-ready intelligence**.



WHAT IT DOES

- Ingests messy JSONL logs (6+ timestamp formats incl. epoch, aliased/missing fields, nested metadata) → canonical schema, recovering **156 unparseable timestamps** and **63 missing IDs** in the sample run.
- 9 channel-aware detectors**: direct / indirect / encoded prompt injection, jailbreak, system-prompt extraction, excessive agency, sensitive-info disclosure (inbound + output), improper output handling / exfil.
- Evasion-resistant**: NFKC + zero-width + spaced-letter + leetspeak folding, and base64 / hex payload decoding before re-checking.
- SQLite store + **7 analyst SQL queries**, incl. an LLM10 consumption-anomaly query and a multi-turn session-escalation correlation.

I RED-TEAMED MY OWN TOOL

An adversarial self-review of the detection code surfaced real bugs — fixed and locked behind tests: a timestamp parser silently turning `20260301` into a 1970 date; a rule firing on every role; unicode / leetspeak / base64 evasions bypassing every regex; and Luhn-less false-positive card matches.

FRAMEWORKS SPOKEN

LLM01 Prompt Injection LLM02 Sensitive Info LLM05 Output Handling LLM06 Excessive Agency LLM07 Prompt Leakage LLM10 Consumption + MITRE ATLAS techniques (AML.T0051 / T0054 / T0056 / T0057 ...).

RESULTS — REPRODUCIBLE RUN (SEED 7)

800 events ingested	150 events flagged	217 findings
83 tests passing	7 analyst SQL queries	9 detectors

BUILT WITH

Python (standard-library-only at runtime) · SQLite · pytest · GitHub Actions CI. Red-team tooling: Microsoft **PyRIT** (runnable offline probe), NVIDIA **garak**, **promptfoo** — plus an offline harness that grades attacks with the same detection engine.

WHY IT MAPS TO THE ROLE (SIA, OPENAI)

- Surface novel harms & abuse patterns** with an ensemble of technical tools and discovery methods — 9 detectors, base64/leetspeak decoding, and a PyRIT/garak/promptfoo red-team loop.
- SQL, Python & AI tooling that improves analysis speed, coverage, and consistency** — a std-lib pipeline, 7 analyst queries, and a tested CLI + CI for repeatable results.
- Structured, decision-ready intelligence** — findings dual-mapped to OWASP/ATLAS with a severity rollup; repeat-offender & session-escalation queries identify, validate, and prioritize risk.
- Anomaly detection for low-prevalence patterns** — consumption-anomaly & multi-turn correlation queries surface rare/novel behavior (a natural fit for PyOD-style tooling).
- A zero-to-one analysis turned into a reusable tool** — built from scratch into a documented, tested, maintainable workflow, not a one-off notebook.